Multi-Agent AI Workflow Engine

Simulating an Autonomous Software Development Team

PROGRESS REPORT · END OF WEEK 4 OF 12

Marco Perozo · Gabriel Marte · Lewdy Aquino · Alvin Almonte
Department of Computer Science · Southwest Minnesota State University

Repository

github.com/marcoaperozo/Capstone-MAWE

Phase 1 complete · Phase 2 in progress · 117 automated tests passing

Where the project stands

Phase 1 is complete and Phase 2 is two thirds done. The engine runs end to end, every run is stored, and three milestone checks run automatically on every push.

Phase	Weeks	Deliverable	Status
1 · Setup & prototyping	1–2	Repository, CI, database schema, pipeline on a state graph	COMPLETE
2 · Role implementation	3–5	Product Manager, Developer and Code Reviewer agents	IN PROGRESS
3 · Memory & state	6–8	Context summarisation, token budget, loop hardening	PLANNED
4 · Testing & evaluation	9–12	Benchmark suite, single-model baseline, final report	PLANNED

Milestone M1 — met

Three-role pipeline runs end to end on a LangGraph state machine, with every exchange persisted.

Milestone M2 — on track

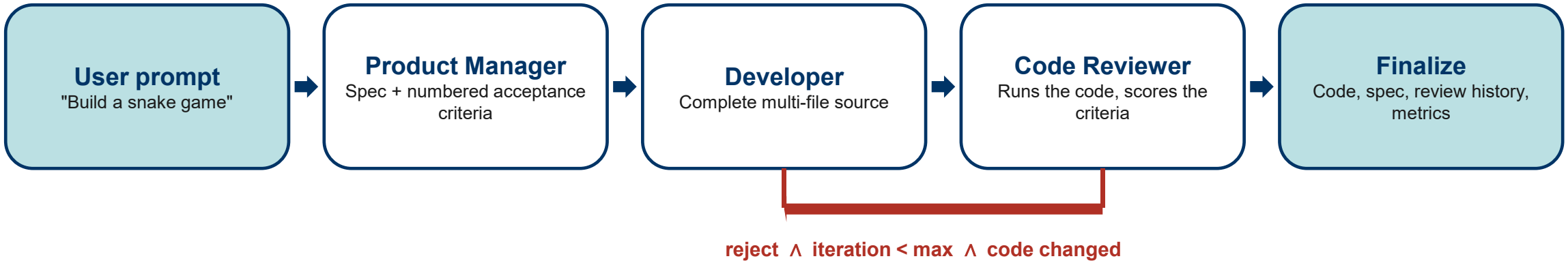
Product Manager and Developer complete and tested. The Code Reviewer is Week 5.

Schedule

On plan. No milestone has slipped and no scope has been dropped.

How the system works

A plain-language prompt is routed through three specialised agents that mirror a real software team. The Reviewer sends work back until it passes or a safety limit stops the loop.



Typed contracts, not prose

Each agent emits structured JSON: the spec, the files, the review verdict. The next agent reads fields, never English. Malformed output is repaired once, then recorded as an error rather than crashing the run.

Everything is recorded

Eleven database tables hold every agent message, every version of the code, every review and every token count. Any past run can be replayed and costed.

Code is executed, not trusted

Generated programs run in a throwaway directory as a separate process with a wall-clock timeout. A crash or a hang is a result the Reviewer reads, never an exception that kills the run.

What was delivered, week by week

Commit history

Each week is a single commit whose message carries that week's milestone result. The history is the progress record.

Week	Deliverable	What it means in practice	Tests
1	Repository, CI, database schema	Package layout, GitHub Actions on three Python versions, all 11 tables with indexes and constraints, architecture document	4
2	State graph and provider layer	LangGraph state machine, four loop-prevention mechanisms, mock and real LLM adapters behind one interface, runs persisted	34
3	Product Manager agent	Specification contract with numbered acceptance criteria, validator, one automatic repair round, five sample prompts	67
4	Developer agent and sandbox	Complete multi-file output, placeholder rejection, subprocess sandbox with timeout and path containment, command-line interface	117

Next: Week 5 completes the Code Reviewer so it scores the acceptance criteria by running tests, rather than reporting what the model claims.

Measured results

Not estimates

Three checks run automatically on every push. Each one executes real code and compares real output — none of these numbers is an assertion.

1 · The loop always terminates, and by the right exit

Scenario	Iterations	How it stopped	Mechanism that fired
Reviewer rejects once, then approves	2	approved	severity threshold
Developer returns identical code	2	no_progress	SHA-256 hash of the generated files
Reviewer never approves	5	iteration_cap	hard cap, default 5

2 · The Product Manager produces usable specs

Check	Result
Valid spec on first answer	5 / 5
Recovered after one repair round	5 / 5

3 · The generated code actually runs

Check	Result
Sandbox executions matching expected output	10 / 10
Multi-file project with cross-module imports	verified

117 automated tests, green on Python 3.11, 3.12 and 3.13. No API key required, zero cost per run.

The core problem: making the loop stop

Research contribution

An agent that reviews another agent's work can reject forever. That is the failure mode that makes autonomous pipelines unusable, and it is the part of this project that is genuinely a computer-science problem rather than an engineering one. Four independent mechanisms guard it.

1 · Hard iteration cap

A configurable ceiling, default five, on review-and-fix cycles. The backstop that always holds.

2 · Severity threshold

Only critical and major issues block approval. A reviewer that objects to formatting cannot stall the pipeline.

3 · No-progress detection

The generated files are hashed. If identical code comes back, the Developer is not responding to feedback and the loop exits immediately.

4 · Fallback routing

Exiting without approval does not fail the run. The best version produced is returned with a warnings report.

Proven, not assumed

The routing policy is one pure function of the run state, so an exhaustive test sweeps every combination of iteration count, severity, verdict and repeated-or-changed code, and asserts that no reachable state can loop forever.

What is working — and what is not yet

Stated plainly so that the remaining risk is visible rather than discovered later.

Component	State	Evidence or what remains
Orchestration — LangGraph state graph	**Working**	Runs end to end; 34 integration tests
Loop prevention — four mechanisms	**Working**	Exhaustive routing sweep; all three exits demonstrated
Persistence — 11 tables, SQLite	**Working**	Every run queryable; PostgreSQL-ready schema
Product Manager agent	**Working**	Valid specs on 5/5 sample prompts, both languages
Developer agent	**Working**	Complete multi-file output; placeholder content rejected
Execution sandbox	**Working**	10/10 executions; timeout, crash and path escape all handled
Code Reviewer agent	**Partial**	Parses the contract and calls the sandbox; does not yet score the criteria by running tests — Week 5
Real-model adapters (Anthropic, OpenAI)	**Untested**	Written behind the same interface; no live API call made yet — needs a key, which is a configuration change, not a code change
Memory management	**Not started**	Weeks 6–8, as scheduled
Benchmark suite and baselines	**Not started**	Weeks 9–12, as scheduled

How to verify any of this

Four commands

Clone the repository and run these. No API key, no network, no cost.

```
pytest -q
```

Runs the full suite: 117 tests in about four seconds.

```
python -m mawe.cli "Build a terminal snake game" --verbose
```

Watch the pipeline live: spec written, code rejected on a named criterion, fixed, approved.

```
python scripts/m1_demo.py --db
```

Three scenarios exiting three different ways, and the database rows they produced.

```
python scripts/week4_dev_check.py
```

Writes generated programs into the sandbox, executes them, compares output.

First-time setup

```
git clone https://github.com/marcoaperozo/Capstone-MAWE.git  &&  cd Capstone-MAWE  &&  pip install -r requirements.txt  &&  pip install -e .
```

Repository: github.com/marcoaperozo/Capstone-MAWE

Full walkthrough in docs/DEMO-GUIDE.md · design rationale in docs/architecture.md